

The Antigravity IDE Master Volume

Complete Exhaustive Documentation: Every Surface, Model, Modality, Tool, All 33 MCP Servers (373 Tools), and All 57 Skills.

CHAPTER 1

The 3 Core AI Modalities in Antigravity IDE

A. Passive Modality: Antigravity Tab Autocomplete & Supercomplete

NEXT-INTENT PREDICTION

Continuous next-intent prediction engine running silently on every keystroke, synthesizing surrounding tabs, terminal outputs, recent diffs, and clipboard context.

- **Tab to Jump:** Anticipates your next navigation point across lines; press `Tab` to jump forward.
- **Tab to Import:** Automatically resolves and inserts missing imports at the top of the file without disrupting cursor line position.
- **Supercomplete:** Displays floating multi-line refactorings and deletions right in your code canvas.
- **Controls:** `Tab` accepts full suggestion, `Esc` cancels, `Ctrl` + `→` accepts word-by-word.

B. Instructive Modality: Inline Command (`Ctrl` + `I` / `Cmd` + `I`)

IN-EDITOR SURGICAL EDITS

Surgical in-place code modification without switching mental context to the chat sidebar.

- **Targeted Refactoring:** Highlight a specific code block, hit `Ctrl` + `I`, and prompt: *"Convert to async/await"* or *"Vectorize with NumPy"*. Only the highlighted lines are touched.
- **Net-New Generation:** Trigger `Ctrl` + `I` on an empty line with a descriptive prompt to generate boilerplate, classes, or test cases directly at the cursor.
- **Instant Documentation:** Highlight any function and trigger `Ctrl` + `I` with *"Add type annotations and Sphinx docstrings"*.

C. Collaborative Modality: Sidebar Chat & Agent Mode

FULL-STACK PAIR PROGRAMMER

The full agentic pair programmer capable of multi-step autonomous reasoning, executing shell commands, reading file trees, diagnosing failures, and managing MCP tools.

- **Inline Code Lenses:** Action triggers rendered directly above classes and functions (*"Explain"*, *"Write Unit Tests"*, *"Refactor"*).
- **Diagnostic Auto-Fix:** Click on compiler errors, lint squiggly lines, or Problems pane entries to invoke the agent directly to analyze and heal the issue.
- **Visual Diff Overlays:** Review inline red/green diffs directly inside your editor canvas before committing changes.

CHAPTER 2

The 7 Exact Models & Reasoning Tiers

Gemini 3.8 Flash (Medium / Low)

⚡ ULTRA FAST DAILY DRIVER

The Daily Workhorse (80% of all programming tasks). Sub-second latency, highest MCP tool execution speed, lowest token burn, and massive context capacity.

Best For: Writing functions, running pytest, CLI benchmarks, high-volume file edits.

Avoid For: Abstract mathematical architecture with 15+ circular dependencies.

Claude Sonnet 5.5 (Medium)

🧠 GOLD STANDARD UI & CLEAN CODE

Unmatched frontend design aesthetics and idiomatic code taste. Produces stunning, modern web apps (CSS/JS) and pristine, typed Python refactors.

Best For: BioNeMo Cockpit UI, modern CSS/JS interfaces, clean architecture.

Avoid For: Trivial shell loops or repetitive regex transforms.

Claude Opus 5.5 (Medium)

🧠 MAXIMUM COGNITIVE DEPTH (S-TIER)

Supreme architectural and mathematical reasoning. Solves novel algorithmic challenges, high-stakes system redesigns, and hard bug autopsies.

Best For: Designing core algorithms from scratch, multi-module pipeline graphs.

Avoid For: Fast back-and-forth chat or budget-limited sessions.

Gemini 3.1 Pro (Low)

🔴 DELIBERATE LOGIC & CONCURRENCY

Deliberate multi-hop reasoning. Traces complex concurrency bugs, race conditions, memory leaks, and interface contracts.

Best For: Diagnosing subtle pipeline crashes, security audits, schema boundaries.

Avoid For: Simple one-line edits or trivial renames.

Gemini 3.7 Flash & 3.6 Flash

⚡ PROVEN STABILITY & BATCH

Proven, stable instruction following. Consistent JSON schema adherence without bleeding-edge drift; great for repetitive batch scripts.

GPT-OSS 120B (Medium)

⚖️ OPEN-WEIGHT SECOND OPINION

Unbiased open-weights inference. Perfect for independent architectural reviews and cross-model code validation.

CHAPTER 3

Chat Canvas, @ Mentions & Slash Commands

- **@Files & Folders** : Attaches file paths so the agent reads surgical slices.
- **@Previous Conversations** : Bridges context from past debugging sessions.
- **@Terminal Sessions** : Injects terminal output without manual copying.
- **@Rules** : Enforces specific coding guidelines on demand.
- **@MCP Tools** : Explicitly targets a specific MCP tool suite.
- **/plan** : Interactive, phased blueprint before touching any code.
- **/goal** : Autonomous endurance mode; runs continuous execution loops until finished.
- **/grill-me** : Interviews you like a Staff Engineer to tease out edge cases.
- **/learn** : Saves corrected rules and workflows into permanent memory across sessions.
- **/schedule** : Configures one-shot timers or recurring cron triggers.

CHAPTER 4

The 17 Core Native Tools

- `view_file` : Reads text, images, PDFs, videos with precise line slices (`StartLine` / `EndLine`).
- `replace_file_content` : Surgically replaces a single contiguous block of code.
- `multi_replace_file_content` : Performs multiple non-contiguous edits atomically.
- `write_to_file` : Creates new files or overwrites existing files.
- `grep_search` : High-speed ripgrep search matching exact strings or regex across files.
- `list_dir` : Recursively lists directory contents and file sizes.
- `run_command` : Executes PowerShell commands in the user's workspace.
- `manage_task` : Controls background processes (`list` , `kill` , `status` , `send_input`).
- `schedule` : Configures asynchronous timers or cron jobs for reactive wakeup.
- `search_web` : Live search engine queries for real-time documentation.
- `read_url_content` : Fetches HTTP content converted directly to Markdown.
- `generate_image` : Generates UI mockups, diagrams, and visual assets.
- `browser_subagent` : Automated browser worker with WebP session recording.
- `ask_question` : Interactive multiple-choice prompt modal in the UI.
- `call_mcp_tool` : Invokes lazy-loaded MCP server endpoints.
- `read_resource` : Reads schemas or persistent state from an MCP server.
- `list_resources` : Lists all available resources exposed by an MCP server.

CHAPTER 5

Complete MCP Server Ecosystem (31 Servers, 373 Tools)

Here is every active Model Context Protocol (MCP) server registered in your environment, along with every tool it exposes and its exact function:

- **apply_design_system** (*assetId, projectId, selectedScreenInstances*): Applies a design system to a list of screens. Use this tool when the user wants to update one or more screens to match the style of a design system. This tool applies the selected design system's foundational design tokens (colors, fonts, shapes, etc.) to the chosen screens, modifying their appearance to align with the design system.
- **create_design_system** (*designSystem, projectId*): Creates a new design system for a project. Use this tool when the user wants to set or update the overall visual theme, style, or branding of the application. This includes configuring:
 - Color Palette: Presets, custom primary colors, and saturation levels.
 - Typography: Font families (e.g., Inter, Roboto, etc.).
 - Shape: Corner roundness for UI elements.
 - Appearance: Light and dark mode background colors.
 - Design MD: Free-form design instructions in markdown.
 This tool establishes the foundational design tokens that apply across all screens in the project. ****Instructions for Tool Call:**** Call ``update_design_system`` tool immediately after this tool to apply the design system to the project, and display the design system in the UI.
- **create_design_system_from_design_md** (*deviceType, projectId, selectedScreenInstance*): Creates a design system for a project, with user uploaded DESIGN.md file, and displays the design system in the UI. ****Instructions for Tool Call:**** Should call ``upload_design_md`` tool first to upload DESIGN.md to a Stitch project.
- **create_project** (*title*): Creates a new Stitch project. A project is a container for UI designs and frontend code.
- **delete_project** (*name*): Deletes a specific Stitch project using its project name. ****Instructions for Tool Call:**** This action cannot be undone. Please confirm with "yes" or "no" to proceed.
- **edit_screens** (*deviceType, modelId, projectId, prompt*): Edits existing screens within a project using a text prompt. ****Instructions for Tool Call:**** This action can take a few minutes to complete. Please be patient. DO NOT RETRY. * If the tool call fails due to connection error, the process may still succeed.
- **generate_screen_from_text** (*designSystem, deviceType, modelId, projectId*): Generates a new screen within a project from a text prompt. ****Instructions for Tool Call:**** This action can take a few minutes to complete. Please be patient. DO NOT RETRY. * If the tool fails with a timeout, don't retry. Instead, try to get the screen with ``get_screen`` method every 30 seconds for up to 10 times before giving up. * If the tool call fails due to connection error, the generation process may still succeed. Please try to get the screen with ``get_screen`` method later. ****Output:**** ``output_components``: If ``output_components`` contains text, return it to the user. If ``output_components`` contains suggestions (e.g. "Yes, make them all"), present these suggestions to the user. If the user accepts one of the suggestions, call ``generate_screen_from_text`` again with ``prompt`` set to the accepted suggestion.
- **generate_variants** (*deviceType, modelId, projectId, prompt*): Generates variants of existing screens within a project using a text prompt. ****Instructions for Tool Call:**** If the tool fails with a timeout, don't retry. Instead, try to get the screen with ``get_screen`` method every 30 seconds for up to 10 times before giving up.
- **get_project** (*name*): Retrieves the details of a specific Stitch project using its project name.
- **get_screen** (*name*): Retrieves the details of a specific screen within a project.
- **list_design_systems** (*projectId*): Lists all design systems for a given project.
- **list_projects** (*filter*): Lists all Stitch projects accessible to the user. By default, it lists projects owned by the user.
- **list_screens** (*projectId*): Lists all screens within a given Stitch project.
- **update_design_system** (*designSystem, name, projectId*): Updates a design system for a project. Use this tool when the user wants to change the overall visual theme, style, or branding of the application. This includes configuring:
 - Color Palette: Presets, custom primary colors, and saturation levels.
 - Typography: Font families (e.g., Inter, Roboto, etc.).
 - Shape: Corner roundness for UI elements.
 - Appearance: Light and dark mode background colors.
 - Design MD: Free-form design instructions in markdown.
 This tool establishes the foundational design tokens that apply across all screens in the project.

- **upload_design_md** (*designMdBase64, projectId*): Uploads DESIGN.md to a Stitch project. Use this tool when the user wants to create a design system from a DESIGN.md file. ****Instructions for Tool Call:**** Call ``create_design_system_from_design_md`` tool immediately after this tool to create the design system from the uploaded DESIGN.md, and display the design system in the UI.

Server: android-management-api

9 TOOLS

- **get_application** (*languageCode, name*): Gets application details for a given enterprise and application ID. Requires the resource name in the format: `enterprises/{enterpriseId}/applications/{applicationId}`.
- **get_device** (*name*): Gets device details for a given enterprise and device ID. Requires the resource name in the format: `enterprises/{enterpriseId}/devices/{deviceId}`.
- **get_enterprise** (*name*): Gets an enterprise for a given enterprise ID. Requires the enterprise ID in the name field (e.g., `enterprises/{enterpriseId}`).
- **get_policy** (*name*): Gets a policy for a given enterprise and policy ID. Requires the resource name in the format: `enterprises/{enterpriseId}/policies/{policyId}`.
- **get_web_app** (*name*): Gets a web app. Requires the resource name in the format: `enterprises/{enterpriseId}/webApps/{webAppId}`.
- **list_devices** (*pageSize, pageToken, parent*): Lists devices for a given enterprise. Requires the enterprise ID in the parent field (e.g., `enterprises/{enterpriseId}`).
- **list_enterprises** (*pageSize, pageToken, projectId, view*): Lists enterprises accessible to the caller.
- **list_policies** (*pageSize, pageToken, parent*): Lists policies for a given enterprise. Requires the enterprise resource name in the parent field (e.g., `enterprises/{enterpriseId}`).
- **list_web_apps** (*pageSize, pageToken, parent*): Lists web apps for a given enterprise. Requires the enterprise resource name in the parent field (e.g., `enterprises/{enterpriseId}`).

Server: arize-tracing-assistant

3 TOOLS

- **arize_support** (*message*): Send **message** to the ``search`` tool and return the assistant reply.
Parameters ----- message : str The user message to send to the assistant. Returns ----- str The assistant's textual response.
- **get_arize_advanced_tracing_docs** (*language*): Get advanced docs and examples to manually instrument an app and send traces/spans to Arize. Parameters -----: language: str "python" or "typescript" or "javascript" Returns: str Docs and code snippets for advanced instrumentation.
- **get_arize_tracing_docs** (*framework, language*): Get docs and examples to instrument an app and send traces/spans to Arize. If the framework is not in the list use manual instrumentation with open telemetry. Parameters ----- framework : str LLM provider or framework. One of: ["agno", "amazon-bedrock", "anthropic", "autogen", "beeai", "crewai", "dspy", "google-gen-ai", "groq", "guardrails-ai", "haystack", "hugging-face-smolagents", "instructor", "langchain", "langflow", "langgraph", "litellm", "llamaindex", "mistralai", "openai", "openai-agents", "prompt-flow", "pydantic-ai", "strands-agents", "together", "vercel", "vertexai"] language : str Programming language: "python" or "typescript" Returns ----- str Example code snippets for auto/manual instrumentation for Arize.

- **cancel_job** (*jobId, location, projectId*): Cancel a running BigQuery job. Use this tool to cancel a query job that is currently executing (i.e. returned ``job_complete: false`` with a ``job_id`` from ``execute_sql`` or ``execute_sql_readonly``). Specify the ``job_id`` to abort.
- **execute_sql** (*dryRun, jobTimeoutMs, labels, projectId*): Run a SQL query in the project and return the result. Prefer the ``execute_sql_readonly`` tool if possible. This tool can execute any query that bigquery supports including:
 - * SQL Queries (``SELECT``, ``INSERT``, ``UPDATE``, ``DELETE``, ``CREATE``, etc.)
 - * AI/ML functions like ``AI.FORECAST``, ``ML.EVALUATE``, ``ML.PREDICT``
 - * Any other query that bigquery supports. Example Queries:


```
``sql -- Insert data into a table. INSERT INTO `my_project.my_dataset`.my_table (name, age) VALUES ('Alice', 30); -- Create a table. CREATE TABLE `my_project.my_dataset`.my_table ( name STRING, age INT64); -- DELETE data from a table. DELETE FROM `my_project.my_dataset`.my_table WHERE name = 'Alice'; -- Create Dataset CREATE SCHEMA `my_project.my_dataset` OPTIONS (location = 'US'); -- Drop table DROP TABLE `my_project.my_dataset`.my_table; -- Drop dataset DROP SCHEMA `my_project.my_dataset`; -- Create Model CREATE OR REPLACE MODEL `my_project.my_dataset.my_model` OPTIONS ( model_type = 'LINEAR_REG' LS_INIT_LEARN_RATE=0.15, L1_REG=1, MAX_ITERATIONS=5, DATA_SPLIT_METHOD='SEQ', DATA_SPLIT_EVAL_FRACTION=0.3, DATA_SPLIT_COL='timestamp') AS SELECT col1, col2, timestamp, label FROM `my_project.my_dataset.my_table`; ``
```

 Queries executed using the ``execute_sql`` tool will always have the default job label ``goog-mcp-server: true`` automatically set in addition to any custom ``labels`` provided in the request. Queries are charged to the project specified in the ``project_id`` field. Query Execution Behavior:
 - * If the query completes within the synchronous timeout (default 20 seconds or custom ``timeout_ms``), the tool returns ``job_complete: true`` and the initial result rows directly. For fast queries, ``job_id`` may be omitted as no persistent background job is created; no further action or polling is needed.
 - * If the query takes longer than ``timeout_ms``, the tool returns ``job_complete: false`` and a ``job_id``. In this case, use the ``get_query_results`` tool with ``job_id`` to poll until ``job_complete: true``, or use ``cancel_job`` to abort the running query.
 - * You can optionally specify ``timeout_ms`` to configure the maximum synchronous wait time in milliseconds (defaults to 20,000 ms), and ``job_timeout_ms`` to enforce a hard server-side timeout after which BigQuery automatically terminates the job.
- **execute_sql_readonly** (*dryRun, jobTimeoutMs, labels, projectId*): Run a read-only SQL query in the project and return the result. Prefer this tool over ``execute_sql`` if possible. This tool is restricted to only ``SELECT`` statements. ``INSERT``, ``UPDATE``, and ``DELETE`` statements and stored procedures aren't allowed. If the query doesn't include a ``SELECT`` statement, an error is returned. For information on creating queries, see the [GoogleSQL documentation](https://cloud.google.com/bigquery/docs/reference/standard-sql/query-syntax). Example Queries:


```
``sql -- Count the number of penguins in each island. SELECT island, COUNT(*) AS population FROM bigquery-public-data.ml_datasets.penguins GROUP BY island -- Evaluate a bigquery ML Model. SELECT * FROM ML.EVALUATE(MODEL `my_dataset.my_model`) -- Evaluate BigQuery ML model on custom data SELECT * FROM ML.EVALUATE(MODEL `my_dataset.my_model`, (SELECT * FROM `my_dataset.my_table`)) -- Predict using BigQuery ML model: SELECT * FROM ML.PREDICT(MODEL `my_dataset.my_model`, (SELECT * FROM `my_dataset.my_table`)) -- Forecast data using AI.FORECAST SELECT * FROM AI.FORECAST(TABLE `project.dataset.my_table`, data_col => 'num_trips', timestamp_col => 'date', id_cols => ['usertype'], horizon => 30) ``
```

 Queries executed using the ``execute_sql_readonly`` tool will always have the job label ``goog-mcp-server: true`` automatically set in addition to any custom ``labels`` provided in the request. Queries are charged to the project specified in the ``project_id`` field. Query Execution Behavior:
 - * If the query completes within the synchronous timeout (default 20 seconds or custom ``timeout_ms``), the tool returns ``job_complete: true`` and the result rows directly. For fast queries, ``job_id`` may be omitted as no persistent background job is created; no further action or polling is needed.
 - * If the query takes longer than ``timeout_ms``, the tool returns ``job_complete: false`` and a ``job_id``. In this case, use the ``get_query_results`` tool with ``job_id`` to poll until ``job_complete: true``, or use ``cancel_job`` to abort the running query.
 - * You can optionally specify ``timeout_ms`` to configure the maximum synchronous wait time in

milliseconds (defaults to 20,000 ms), and ``job_timeout_ms`` to enforce a hard server-side timeout after which BigQuery automatically terminates the job.

- `get_dataset_info` (*datasetId, projectId*): Get metadata information about a BigQuery dataset or BigLake namespace.
- `get_query_results` (*jobId, location, maxResults, pageToken*): Get the results of a BigQuery SQL query job. Use this tool ONLY when: 1. A previous ``execute_sql`` or ``execute_sql_readonly`` call returned ``job_complete: false`` with a ``job_id`` (poll with this tool until ``job_complete: true``), OR 2. You need to paginate through additional rows using ``page_token`` or ``start_index`` for a previously completed job. Do NOT call this tool if the query already returned ``job_complete: true`` with all rows. Supports pagination. Use ``max_results`` to limit results and ``page_token`` to retrieve the next page of results.
- `get_table_info` (*datasetId, projectId, tableId*): Get metadata information about a BigQuery table or BigLake table.
- `list_dataset_ids` (*pageSize, pageToken, projectId*): List BigQuery dataset IDs and BigLake namespaces in a Google Cloud project. Supports pagination. Use ``page_size`` to limit results and ``page_token`` to retrieve next page.
- `list_table_ids` (*datasetId, pageSize, pageToken, projectId*): List table ids in a BigQuery dataset or BigLake namespace. Supports pagination. Use ``page_size`` to limit results and ``page_token`` to retrieve next page.

- **bionemo_compare_known_inhibitors** (*candidate_affinity_kcal_mol, target_name*): Benchmark AI-generated candidate molecules against clinically approved reference drugs. Args: target_name: Target identifier or disease (e.g. 'EGFR', 'Mpro', 'HER2', 'KRAS', 'PARP1'). candidate_affinity_kcal_mol: Optional predicted affinity of candidate (in kcal/mol, e.g. -9.5). Returns: Clinical reference drug benchmarks with IC50, experimental affinity, and comparative score.
- **bionemo_fetch_rcsb_pdb** (*pdb_id*): Fetch, clean, and validate macromolecular structures directly from RCSB PDB. Strips solvent, crystallographic waters, and isolates canonical protein chains. Args: pdb_id: 4-character PDB code (e.g. '2ITZ', '6LU7', '8AZV', '4MNE'). Returns: Dictionary with status, clean PDB atom count, residue count, and extracted sequence.
- **bionemo_query_diffdock** (*ligand_smiles, mock, pdb_content, protein_sequence*): Predict small molecule binding poses and affinities using NVIDIA NIM DiffDock. Args: protein_sequence: Amino acid sequence of the target protein. ligand_smiles: SMILES string of the small molecule drug/ligand. pdb_content: Optional raw PDB file string. If not provided, a synthetic backbone is generated. mock: If True, uses simulated realistic response. If False, calls live NIM endpoint. Returns: Docking results including top affinity (kcal/mol), number of poses, and latency profile.
- **bionemo_query_esm2_embedding** (*mock, sequence*): Query NVIDIA NIM ESM-2 (650M) for per-residue embeddings. Args: sequence: Valid IUPAC protein amino acid sequence. mock: If True, uses the realistic simulation mock (zero API credits required). If False, calls live NVIDIA NIM microservice using NVIDIA_API_KEY. Returns: Summary of sequence length, hidden dimension, latency breakdown, and embedding shape.
- **bionemo_query_esmfold** (*mock, sequence*): Predict atomic 3D protein structure from amino acid sequence via NVIDIA BioNeMo ESMFold NIM. Args: sequence: IUPAC protein amino acid sequence. mock: If True, uses zero-credit realistic simulation with predicted pLDDT confidence. If False, calls live NVIDIA NIM ESMFold endpoint. Returns: Dictionary with predicted structure status, average pLDDT confidence, and atom records.
- **bionemo_run_benchmark** (*complexes_path, mock, save_plots*): Execute the NVIDIA BioNeMo & NIM Inference Benchmark Suite across the sample complexes. Args: mock: If True, executes using zero-configuration simulated responses. If False, connects to live NVIDIA NIM endpoints using NVIDIA_API_KEY. complexes_path: Path to JSON file containing complexes (default: data/sample_complexes.json). save_plots: Whether to save visualization PNG charts to results/. Returns: Consolidated summary metrics table, speedup factors, and throughput.
- **bionemo_sanitize_smiles** (*smiles*): Validate, sanitize, and extract physicochemical descriptors for a small molecule SMILES before dispatching to NVIDIA NIM DiffDock or MolMIM. Args: smiles: Chemical structure SMILES string. Returns: Dictionary containing canonical SMILES, Molecular Weight, LogP, TPSA, HBD, HBA.
- **bionemo_validate_sequence** (*sequence*): Validate and profile a protein amino acid sequence for NVIDIA BioNeMo / ESM-2. Args: sequence: Single-letter amino acid sequence (IUPAC format). Returns: Dictionary with validation status, clean sequence, length, and residue frequency.

- **click** (*dblClick, includeSnapshot, pageId, uid*): Clicks on the provided element
- **close_page** (*pageId*): Closes the page by its index. The last open page cannot be closed.
- **drag** (*from_uid, includeSnapshot, pageId, to_uid*): Drag an element onto another element
- **emulate** (*colorScheme, cpuThrottlingRate, extraHttpHeaders, geolocation*): Emulates various features on the target page.
- **evaluate_script** (*args, dialogAction, filePath, function*): Evaluate a JavaScript function inside the target page. Returns the response as JSON, so returned values have to be JSON-serializable.
- **fill** (*includeSnapshot, pageId, uid, value*): Type text into an input, text area or select an option from a 